Import Libraries

In [1]:
```python
import pandas as pd
import numpy as np


import matplotlib.pyplot as plt
import seaborn as sns

import sklearn
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score

import joblib
import warnings

warnings.filterwarnings("ignore")

print("Libraries imported successfully")
```

```
Libraries imported successfully
```

In [2]:
```python
df=pd.read_csv("D:\Downloads\clients.csv")              # Load the client dataset

print("Dataset loaded successfully")
```

```
Dataset loaded successfully
```

In [3]:
```python
df.head()
```

Out[3]:

	client_id	client_type	first_name	last_name	date_of_birth	gender	country	region	acquisition_purpose	satisfaction_score	loan_applied	referral_channel
0	C0001	Individual	Kareem	Liu	05-11-1968	F	USA	California	Home	4	Yes	Website
1	C0002	Individual	Trystan	Oconnor	11/26/1962	M	USA	California	Home	1	No	Website
2	C0003	Individual	Kale	Gay	04-07-1959	M	USA	California	Home	4	Yes	Agency
3	C0004	Individual	Russell	Gross	11/25/1959	M	USA	California	Home	5	No	Website
4	C0005	Company	Marleez	Co	2/28/1976	M	USA	California	Investment	5	No	Website

In [4]:
```python
property_df=pd.read_csv("D:\Downloads\properties.csv")      # Load the property dataset

print(" successfully load")
```

```
 successfully load
```

In [5]:
```python
property_df.head()
```

Out[5]:

	listing_id	tower_number	transaction_date	unit_category	unit_number	floor_area_sqft	sale_price	listing_status	client_ref
0	1012	1	01-01-2024	Apartment	12	1160.36	$300,385.62	Sold	C0027
1	1015	1	01-01-2024	Apartment	15	782.25	$208,930.81	Sold	C0097
2	1021	1	01-01-2024	Apartment	21	756.21	$218,585.92	Sold	C0113
3	1030	1	01-01-2024	Apartment	30	743.09	$246,172.68	Sold	C0141
4	2016	2	01-01-2024	Apartment	16	701.66	$212,265.67	Sold	C0146

Dataset Information

In [6]:
```python
# check client dataset dimensions
print(df.shape)
```

```
(2000, 12)
```

In [7]:
```python
# check property dataset dimensions
print(property_df.shape)
```

```
(10000, 9)
```

In [8]:
```python
# Display column names
print("client_columns=", df.columns)

print("property_columns=", property_df.columns)
```

```
client_columns= Index(['client_id', 'client_type', 'first_name', 'last_name', 'date_of_birth',
       'gender', 'country', 'region', 'acquisition_purpose',
       'satisfaction_score', 'loan_applied', 'referral_channel'],
      dtype='object')
property_columns= Index(['listing_id', 'tower_number', 'transaction_date', 'unit_category',
       'unit_number', 'floor_area_sqft', 'sale_price', 'listing_status',
       'client_ref'],
      dtype='object')
```

In [9]:
```python
# check data types

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 12 columns):
 #   Column               Non-Null Count  Dtype 
---  ------               --------------  ----- 
 0   client_id            2000 non-null   object
 1   client_type          2000 non-null   object
 2   first_name           2000 non-null   object
 3   last_name            2000 non-null   object
 4   date_of_birth        2000 non-null   object
 5   gender               2000 non-null   object
 6   country              2000 non-null   object
 7   region               2000 non-null   object
 8   acquisition_purpose  2000 non-null   object
 9   satisfaction_score   2000 non-null   int64 
 10  loan_applied         2000 non-null   object
 11  referral_channel     2000 non-null   object
dtypes: int64(1), object(11)
memory usage: 187.6+ KB
```

In [10]:
```python
# check data types
property_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   listing_id        10000 non-null  int64  
 1   tower_number      10000 non-null  int64  
 2   transaction_date  10000 non-null  object 
 3   unit_category     10000 non-null  object 
 4   unit_number       10000 non-null  int64  
 5   floor_area_sqft   10000 non-null  float64
 6   sale_price        10000 non-null  object 
 7   listing_status    10000 non-null  object 
 8   client_ref        7305 non-null   object 
dtypes: float64(1), int64(3), object(5)
memory usage: 703.3+ KB
```

In [11]:
```python
# Statistical Summary
df.describe()
```

Out[11]:

	satisfaction_score
count	2000.000000
mean	3.029000
std	1.413562
min	1.000000
25%	2.000000
50%	3.000000
75%	4.000000
max	5.000000

In [12]:
```python
# Statistical summary
property_df.describe()
```

Out[12]:

	listing_id	tower_number	unit_number	floor_area_sqft
count	10000.000000	10000.000000	10000.00000	10000.000000
mean	105898.277400	10.370100	28.99920	1139.941412
std	74414.566199	5.769025	16.88242	418.373967
min	1002.000000	1.000000	1.00000	410.710000
25%	50389.750000	5.000000	15.00000	782.200000
50%	100404.500000	10.000000	29.00000	1110.880000
75%	150411.250000	15.000000	43.00000	1499.000000
max	990026.000000	20.000000	70.00000	1957.160000

In [13]:
```python
# check missing values

print(df.isnull().sum())
```

```
client_id              0
client_type            0
first_name             0
last_name              0
date_of_birth          0
gender                 0
country                0
region                 0
acquisition_purpose    0
satisfaction_score     0
loan_applied           0
referral_channel       0
dtype: int64
```

In [14]:
```python
# check missing values
property_df.isnull().sum()
```

Out[14]:
```
listing_id             0
tower_number           0
transaction_date       0
unit_category          0
unit_number            0
floor_area_sqft        0
sale_price             0
listing_status         0
client_ref          2695
dtype: int64
```

Data Cleaning

In [15]:
```python
# check duplicate client records

print(" Duplicate client records:",df.duplicated().sum())

# df=df.drop_duplicates()
# print("shape after removing duplicates:",df.shape)
```

```
 Duplicate client records: 0
```

In [16]:
```python
# check duplicate property records
print("Duplicate property records:",property_df.duplicated().sum())
```

```
Duplicate property records: 0
```

In [17]:
```python
# check duplicate client IDs
print("duplicates client IDs:",df["client_id"].duplicated().sum())
```

```
duplicates client IDs: 0
```

In [18]:
```python
# convert date of birth into age

df["date_of_birth"]=pd.to_datetime(df["date_of_birth"],format="mixed",dayfirst=False)         # Convert DOB into age

today=pd.Timestamp.today()

df["age"]=(today.year-df["date_of_birth"].dt.year)
```

In [19]:
```python
df.head()
```

Out[19]:

	client_id	client_type	first_name	last_name	date_of_birth	gender	country	region	acquisition_purpose	satisfaction_score	loan_applied	referral_channel	age
0	C0001	Individual	Kareem	Liu	1968-05-11	F	USA	California	Home	4	Yes	Website	58
1	C0002	Individual	Trystan	Oconnor	1962-11-26	M	USA	California	Home	1	No	Website	64
2	C0003	Individual	Kale	Gay	1959-04-07	M	USA	California	Home	4	Yes	Agency	67
3	C0004	Individual	Russell	Gross	1959-11-25	M	USA	California	Home	5	No	Website	67
4	C0005	Company	Marleez	Co	1976-02-28	M	USA	California	Investment	5	No	Website	50

In [20]:
```python
df["age"].describe()
```

Out[20]:
```
count    2000.000000
mean       56.099000
std        17.343719
min        26.000000
25%        41.000000
50%        57.000000
75%        71.000000
max        95.000000
Name: age, dtype: float64
```

In [21]:
```python
#  Clean categorical Values

categorical_columns = ["client_type","gender","country","region","acquisition_purpose","loan_applied","referral_channel"]

for col in categorical_columns:
    df[col] = (df[col].astype(str).str.strip().str.title())
```

In [22]:
```python
for col in categorical_columns:
    print(col)
```

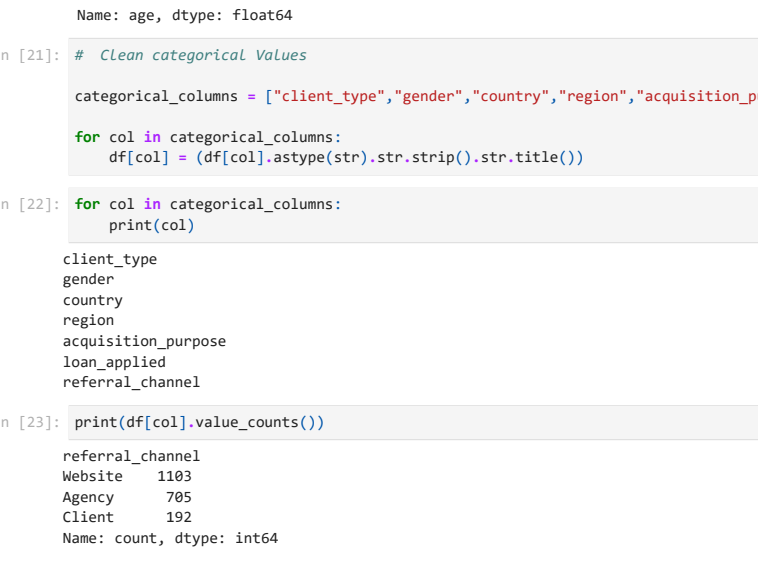
```
client_type
gender
country
region
acquisition_purpose
loan_applied
referral_channel
```

In [23]:
```python
print(df[col].value_counts())
```

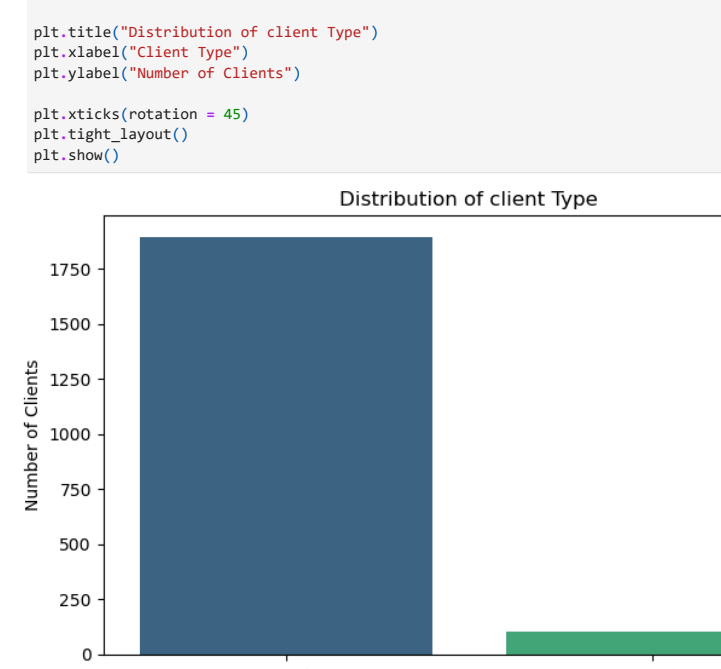
```
referral_channel
Website    1103
Agency      705
Client      192
Name: count, dtype: int64
```

EDA (Exploratory Data Analysis)

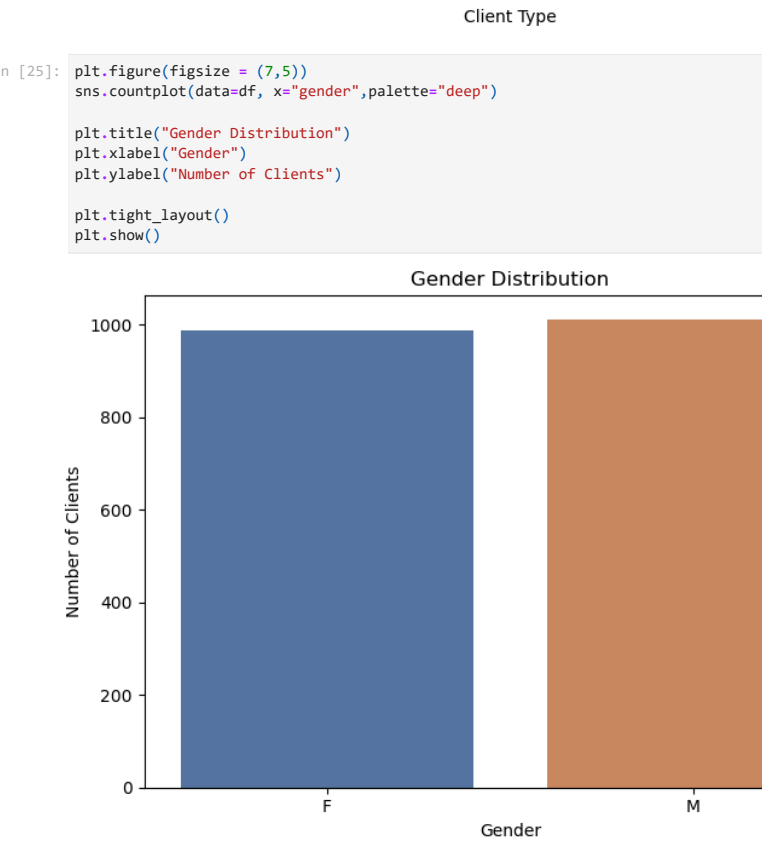
In [24]:
```python
plt.figure(figsize=(7,5))
sns.countplot(data = df, x="client_type",palette="viridis")

plt.title("Distribution of client Type")
plt.xlabel("Client Type")
plt.ylabel("Number of Clients")

plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()
```



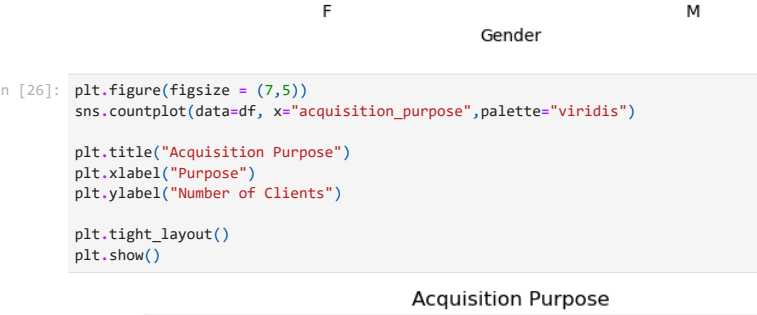
In [25]:
```python
plt.figure(figsize = (7,5))
sns.countplot(data=df, x="gender",palette="deep")

plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Clients")

plt.tight_layout()
plt.show()
```



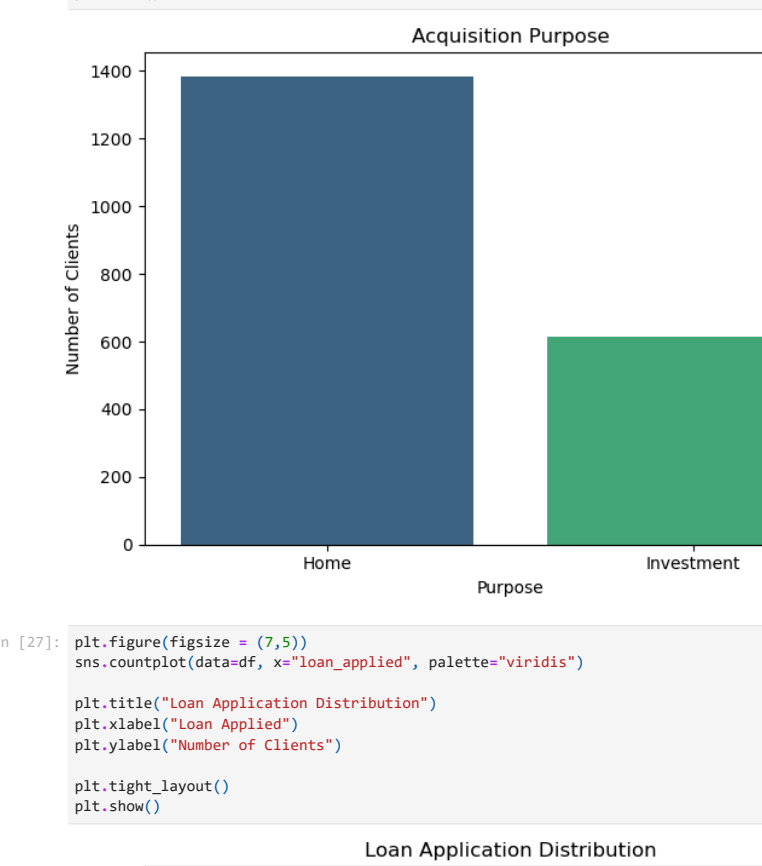
In [26]:
```python
plt.figure(figsize = (7,5))
sns.countplot(data=df, x="acquisition_purpose",palette="viridis")

plt.title("Acquisition Purpose")
plt.xlabel("Purpose")
plt.ylabel("Number of Clients")

plt.tight_layout()
plt.show()
```



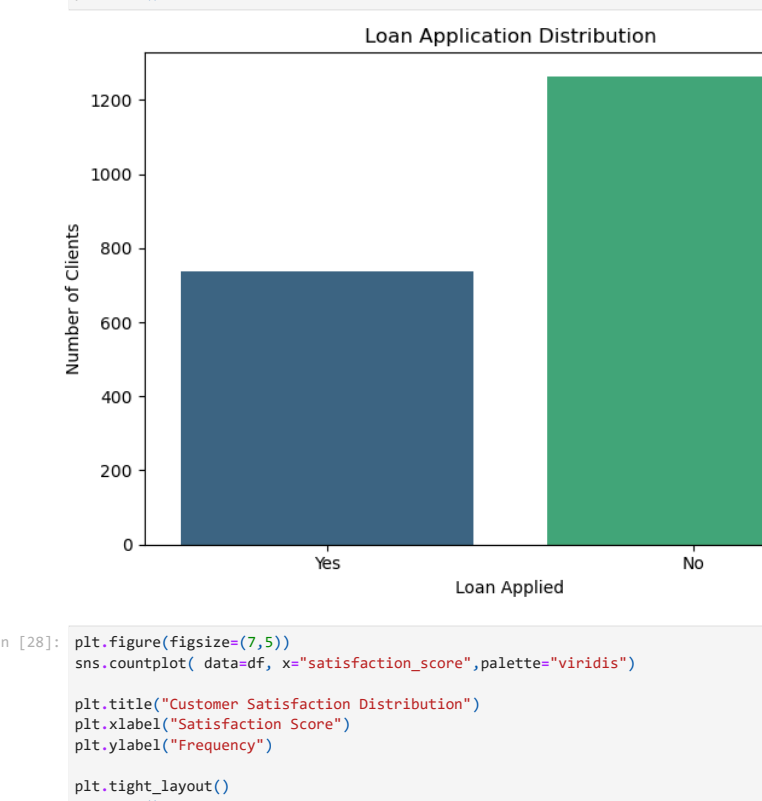
In [27]:
```python
plt.figure(figsize = (7,5))
sns.countplot(data=df, x="loan_applied", palette="viridis")

plt.title("Loan Application Distribution")
plt.xlabel("Loan Applied")
plt.ylabel("Number of Clients")

plt.tight_layout()
plt.show()
```



In [28]:
```python
plt.figure(figsize=(7,5))
sns.countplot( data=df, x="satisfaction_score",palette="viridis")

plt.title("Customer Satisfaction Distribution")
plt.xlabel("Satisfaction Score")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



In [29]:
```python
plt.figure(figsize=(7,5))
sns.histplot(data=df, x="age",kde=True, color="purple")

plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



In [30]:
```python
ml_df = df.copy()

ml_df = ml_df.drop(columns = ["client_id","first_name","last_name","date_of_birth"])
print(ml_df.head())
```

```
  client_type gender country      region acquisition_purpose  \
0  Individual      F     Usa  California                Home   
1  Individual      M     Usa  California                Home   
2  Individual      M     Usa  California                Home   
3  Individual      M     Usa  California                Home   
4     Company      M     Usa  California          Investment   

   satisfaction_score loan_applied referral_channel  age  
0                   4          Yes          Website   58  
1                   1           No          Website   64  
2                   4          Yes           Agency   67  
3                   5           No          Website   67  
4                   5           No          Website   50  
```

Feature Engineering

In [31]:
```python
# handle missing values

numeric_columns = ["age","satisfaction_score"]

for col in numeric_columns:
    ml_df[col] = ml_df[col].fillna(ml_df[col].median())
```

In [32]:
```python
# categorical columns
for col in categorical_columns:
    if col in ml_df.columns:
        ml_df[col] = ml_df[col].fillna(ml_df[col].mode()[0])
```

In [33]:
```python
numeric_features = ["age","satisfaction_score"]

categorical_features = ["client_type","gender","country","region","acquisition_purpose","loan_applied","referral_channel"]
```

In [34]:
```python
# One-Hot Encoding

preprocessor = ColumnTransformer(transformers = [("num", StandardScaler(), numeric_features),
                                                  ("cat", OneHotEncoder(handle_unknown="ignore"),categorical_features)])
```

In [35]:
```python
# Transform the dataset
x = preprocessor.fit_transform(ml_df)

print("Feature matrix created!")
print("shape:",x.shape)
```

```
Feature matrix created!
shape: (2000, 80)
```

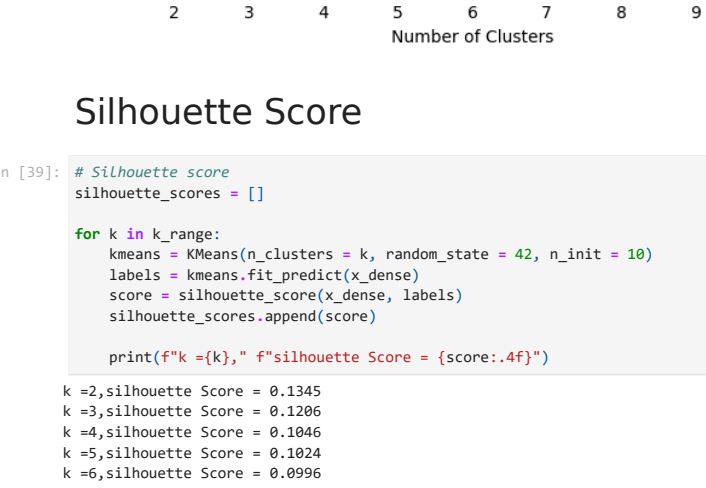
In [36]:
```python
# convert sparse matrix if necessary

x_dense = x.toarray()

print(x_dense.shape)
```

```
(2000, 80)
```

Elbow Method

In [37]:
```python
# find optimal K using Elbow Method

inertia = []
k_range = range(2,11)

for k in k_range:
    kmeans = KMeans(n_clusters = k, random_state = 42, n_init = 10)
    kmeans.fit(x_dense)
    inertia.append(kmeans.inertia_)
```

In [38]:
```python
plt.figure(figsize = (7,5))
plt.plot(k_range, inertia, marker = "o",color="blue")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")

plt.xticks(k_range)
plt.grid(True)

plt.show()
```

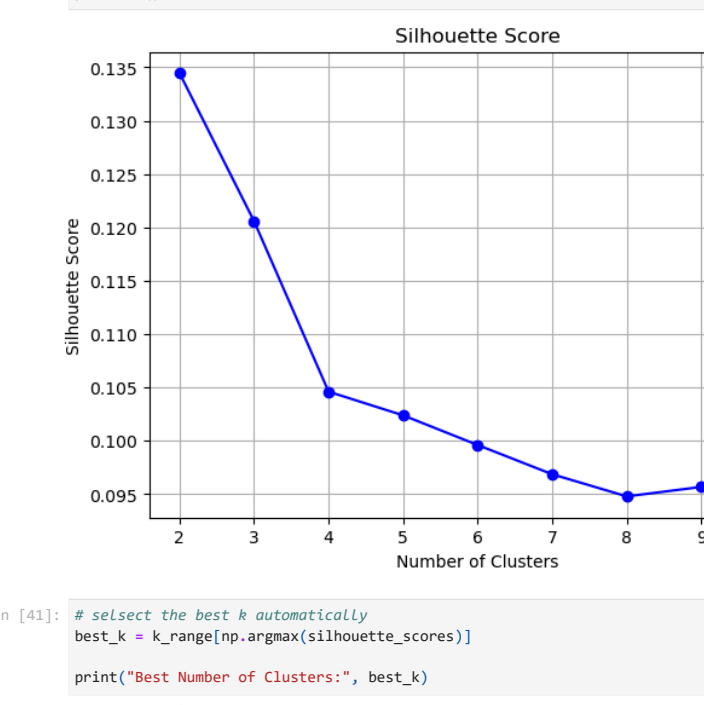


Silhouette Score

In [39]:
```python
# Silhouette score
silhouette_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters = k, random_state = 42, n_init = 10)
    labels = kmeans.fit_predict(x_dense)
    score = silhouette_score(x_dense, labels)
    silhouette_scores.append(score)

    print(f"k ={k}," f"silhouette Score = {score:.4f}")
```

```
k =2,silhouette Score = 0.1345
k =3,silhouette Score = 0.1206
k =4,silhouette Score = 0.1046
k =5,silhouette Score = 0.1024
k =6,silhouette Score = 0.0996
k =7,silhouette Score = 0.0969
k =8,silhouette Score = 0.0948
k =9,silhouette Score = 0.0957
k =10,silhouette Score = 0.0969
```

In [40]:
```python
plt.figure(figsize=(7,5))
plt.plot(k_range, silhouette_scores, marker="o",color="blue")
plt.title("Silhouette Score")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")

plt.xticks(k_range)
plt.grid(True)

plt.show()
```



In [41]:
```python
# selsect the best K automatically
best_k = k_range[np.argmax(silhouette_scores)]

print("Best Number of Clusters:", best_k)
```

```
Best Number of Clusters: 2
```

KMeans Cluster

In [42]:
```python
# apply K-Means
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(x_dense)

print(cluster_labels[:20])
```

```
[1 0 1 1 1 1 1 1 1 0 1 0 1 1 1 1 0 1 1 1]
```

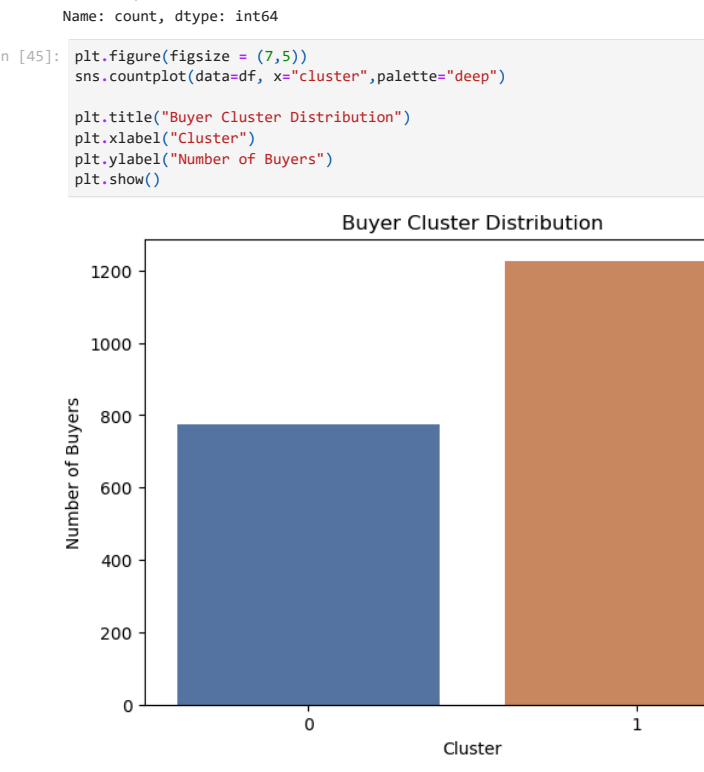
In [43]:
```python
# add cluster labels to dataset
df["cluster"] = cluster_labels

print(df[["client_id","cluster"]].head())
```

```
  client_id  cluster
0     C0001        1
1     C0002        0
2     C0003        1
3     C0004        1
4     C0005        1
```

In [44]:
```python
# Cluster distribution
print(df["cluster"].value_counts().sort_index())
```

```
cluster
0     774
1    1226
Name: count, dtype: int64
```

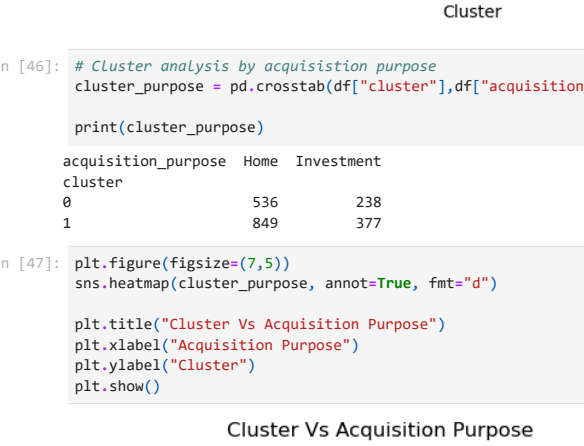
In [45]:
```python
plt.figure(figsize = (7,5))
sns.countplot(data=df, x="cluster",palette="deep")

plt.title("Buyer Cluster Distribution")
plt.xlabel("Cluster")
plt.ylabel("Number of Buyers")
plt.show()
```



In [46]:
```python
# Cluster analysis by acquisition purpose
cluster_purpose = pd.crosstab(df["cluster"],df["acquisition_purpose"])

print(cluster_purpose)
```

```
acquisition_purpose  Home  Investment
cluster                              
0                     536         238
1                     849         377
```

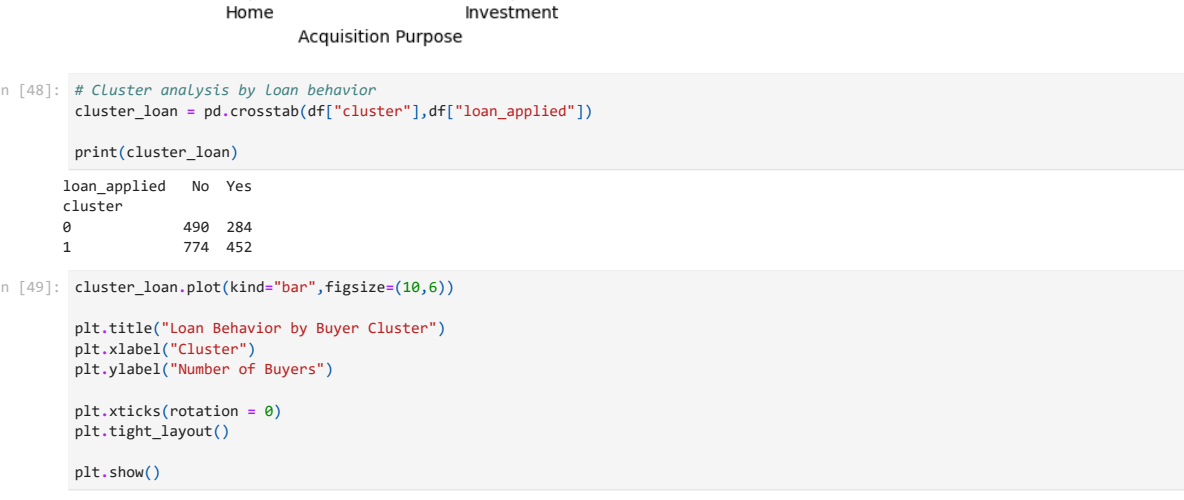
In [47]:
```python
plt.figure(figsize=(7,5))
sns.heatmap(cluster_purpose, annot=True, fmt="d")

plt.title("Cluster Vs Acquisition Purpose")
plt.xlabel("Acquisition Purpose")
plt.ylabel("Cluster")
plt.show()
```



In [48]:
```python
# Cluster analysis by loan behavior
cluster_loan = pd.crosstab(df["cluster"],df["loan_applied"])

print(cluster_loan)
```

```
loan_applied   No  Yes
cluster               
0             490  284
1             774  452
```

In [49]:
```python
cluster_loan.plot(kind="bar",figsize=(10,6))

plt.title("Loan Behavior by Buyer Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Buyers")

plt.xticks(rotation = 0)
plt.tight_layout()

plt.show()
```



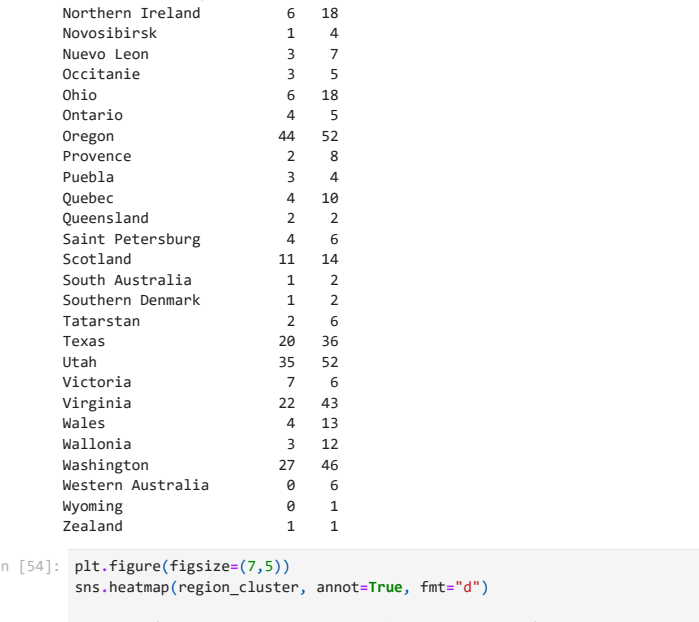
In [50]:
```python
# average age by cluster
age_cluster = (df.groupby("cluster")["age"].mean())

print(age_cluster)
```

```
cluster
0    55.684755
1    56.360522
Name: age, dtype: float64
```

In [51]:
```python
# satisfaction by cluster
satisfaction_cluster = (df.groupby("cluster")["satisfaction_score"].mean())

print(satisfaction_cluster)
```

```
cluster
0    1.489664
1    4.000816
Name: satisfaction_score, dtype: float64
```

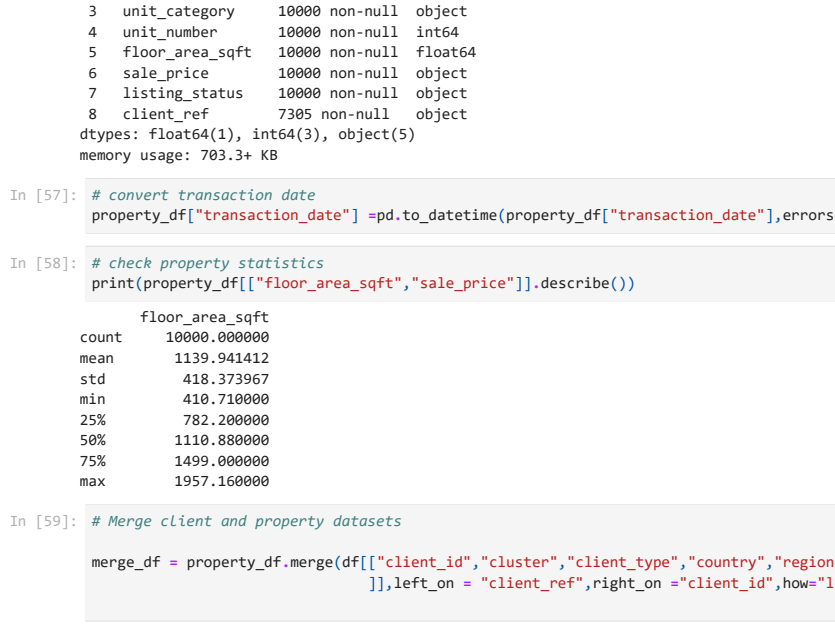
In [52]:
```python
plt.figure(figsize=(7,5))
sns.barplot(x=satisfaction_cluster.index, y=satisfaction_cluster.values,palette="viridis")

plt.title("Average Satisfaction Score by Cluster")
plt.xlabel("Cluster")
plt.ylabel("Average Satisfaction")

plt.show()
```



In [53]:
```python
# Geographic analysis
region_cluster = pd.crosstab(df["region"], df["cluster"])

print(region_cluster)
```

```
cluster                  0    1
region                         
Alberta                 10   17
Arizona                 37   71
Baja California          3    4
Bavaria                  3    9
Berlin                   4    7
British Columbia         7   13
Brittany                 7    9
Brussels                 7    5
California             255  378
Capital Region           4    1
Central Denmark          1    0
Colorado                46   72
England                 13   16
Flanders                 9    7
Florida                 23   27
Georgia                 11   23
Hamburg                  7    9
Hesse                    1    6
Ile-De-France            5   10
Jalisco                  4    5
Kansas                   0    1
Krasnodar Krai           6    4
Manitoba                 4   11
Mexico City              0    7
Moscow Oblast            2    1
Nevada                  57   86
New South Wales          6    7
New York                18   31
Normandy                 2    2
North Denmark            2    2
North Rhine-Westphalia   4    6
Northern Ireland         6   18
Novosibirsk              1    4
Nuevo Leon               3    7
Occitanie                3    5
Ohio                     6   18
Ontario                  4    5
Oregon                  44   52
Provence                 2    8
Puebla                   3    4
Quebec                   4   10
Queensland               2    2
Saint Petersburg         4    6
Scotland                11   14
South Australia          1    2
Southern Denmark         1    2
Tatarstan                2    6
Texas                   20   36
Utah                    35   52
Victoria                 7    6
Virginia                22   43
Wales                    4   13
Wallonia                 3   12
Washington              27   46
Western Australia        0    6
Wyoming                  0    1
Zealand                  1    1
```

In [54]:
```python
plt.figure(figsize=(7,5))
sns.heatmap(region_cluster, annot=True, fmt="d")

plt.title("Geographic Distribution of Buyer Clusters")
plt.xlabel("Cluster")
plt.ylabel("Region")

plt.show()
```



In [55]:
```python
# Analyze property data
property_df.head()
```

Out[55]:

	listing_id	tower_number	transaction_date	unit_category	unit_number	floor_area_sqft	sale_price	listing_status	client_ref
0	1012	1	01-01-2024	Apartment	12	1160.36	$300,385.62	Sold	C0027
1	1015	1	01-01-2024	Apartment	15	782.25	$208,930.81	Sold	C0097
2	1021	1	01-01-2024	Apartment	21	756.21	$218,585.92	Sold	C0113
3	1030	1	01-01-2024	Apartment	30	743.09	$246,172.68	Sold	C0141
4	2016	2	01-01-2024	Apartment	16	701.66	$212,265.67	Sold	C0146

In [56]:
```python
property_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   listing_id        10000 non-null  int64  
 1   tower_number      10000 non-null  int64  
 2   transaction_date  10000 non-null  object 
 3   unit_category     10000 non-null  object 
 4   unit_number       10000 non-null  int64  
 5   floor_area_sqft   10000 non-null  float64
 6   sale_price        10000 non-null  object 
 7   listing_status    10000 non-null  object 
 8   client_ref        7305 non-null   object 
dtypes: float64(1), int64(3), object(5)
memory usage: 703.3+ KB
```

In [57]:
```python
# convert transaction date
property_df["transaction_date"] =pd.to_datetime(property_df["transaction_date"],errors="coerce")
```

In [58]:
```python
# check property statistics
print(property_df[["floor_area_sqft","sale_price"]].describe())
```

```
       floor_area_sqft
count     10000.000000
mean       1139.941412
std         418.373967
min         410.710000
25%         782.200000
50%        1110.880000
75%        1499.000000
max        1957.160000
```

In [59]:
```python
# Merge client and property datasets

merge_df = property_df.merge(df[["client_id","cluster","client_type","country","region","acquisition_purpose","loan_applied","satisfaction_score","age"
                                ]],left_on = "client_ref",right_on ="client_id",how="left")
```

In [60]:
```python
print(merge_df.head())
print(merge_df.shape)
```

```
   listing_id  tower_number transaction_date unit_category  unit_number  \
0        1012             1       2024-01-01     Apartment           12   
1        1015             1       2024-01-01     Apartment           15   
2        1021             1       2024-01-01     Apartment           21   
3        1030             1       2024-01-01     Apartment           30   
4        2016             2       2024-01-01     Apartment           16   

   floor_area_sqft   sale_price listing_status client_ref client_id  cluster  \
0          1160.36  $300,385.62           Sold      C0027     C0027      1.0   
1           782.25  $208,930.81           Sold      C0097     C0097      1.0   
2           756.21  $218,585.92           Sold      C0113     C0113      1.0   
3           743.09  $246,172.68           Sold      C0141     C0141      1.0   
4           701.66  $212,265.67           Sold      C0146     C0146      0.0   

  client_type country      region acquisition_purpose loan_applied  \
0  Individual     Usa  California                Home          Yes   
1  Individual     Usa    Virginia                Home          Yes   
2  Individual     Usa        Utah          Investment           No   
3  Individual     Usa  California                Home           No   
4  Individual     Usa    Colorado                Home           No   

   satisfaction_score   age  
0                 5.0  58.0  
1                 5.0  83.0  
2                 4.0  77.0  
3                 5.0  40.0  
4                 2.0  63.0  
(10000, 18)
```

In [61]:
```python
# Investment value by cluster
investment_by_cluster = (merge_df.groupby("cluster")["sale_price"].sum())

print(investment_by_cluster)
```

```
cluster
0.0    $212,265.67$240,539.35$336,600.53$232,423.05$2...
1.0    $300,385.62$208,930.81$218,585.92$246,172.68$2...
Name: sale_price, dtype: object
```

In [62]:
```python
# Convert sale_price into numeric

merge_df["sale_price"] = (
    merge_df["sale_price"]
    .astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.strip()
)

merge_df["sale_price"] = pd.to_numeric(
    merge_df["sale_price"],
    errors="coerce"
)

# Check datatype
print(merge_df["sale_price"].dtype)
```

```
float64
```

In [63]:
```python
# average property price by cluster
avg_price = (merge_df.groupby("cluster")["sale_price"].mean())
print(avg_price)
```

```
cluster
0.0    344716.680677
1.0    345291.693635
Name: sale_price, dtype: float64
```

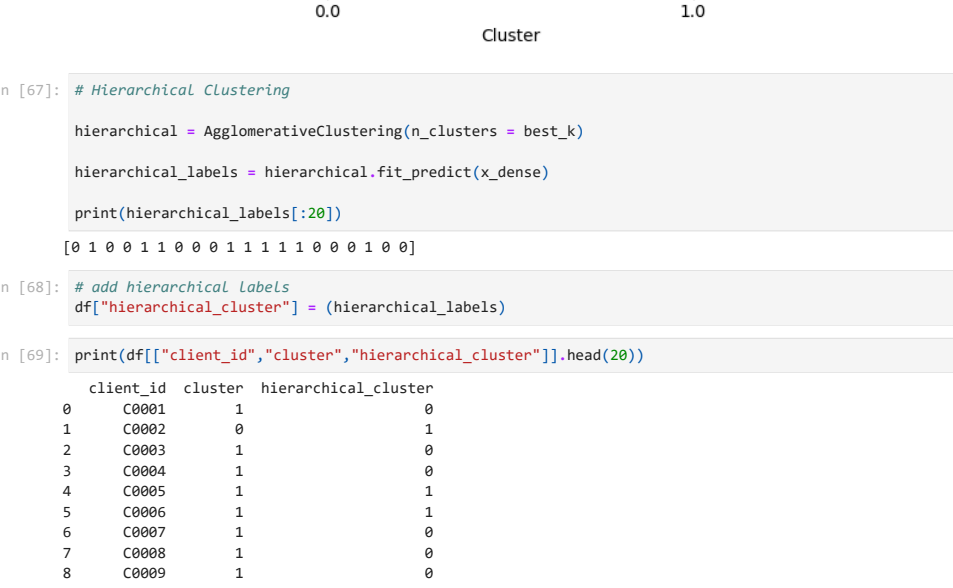
In [64]:
```python
# Property area by cluster
avg_price = (merge_df.groupby("cluster")["floor_area_sqft"].mean())
print(avg_price)
```

```
cluster
0.0    1139.383963
1.0    1142.238440
Name: floor_area_sqft, dtype: float64
```

In [65]:
```python
# Property Category by cluster
category_cluster = pd.crosstab(merge_df["cluster"],merge_df["unit_category"])
print(category_cluster)
```

```
unit_category  Apartment  Office
cluster                         
0.0                 2380     411
1.0                 3845     669
```

In [66]:
```python
category_cluster.plot(kind="bar",figsize=(7,5))

plt.title("property Category by Buyer Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Transactions")

plt.xticks(rotation=0)
plt.tight_layout()

plt.show()
```



In [67]:
```python
# Hierarchical Clustering

hierarchical = AgglomerativeClustering(n_clusters = best_k)

hierarchical_labels = hierarchical.fit_predict(x_dense)

print(hierarchical_labels[:20])
```

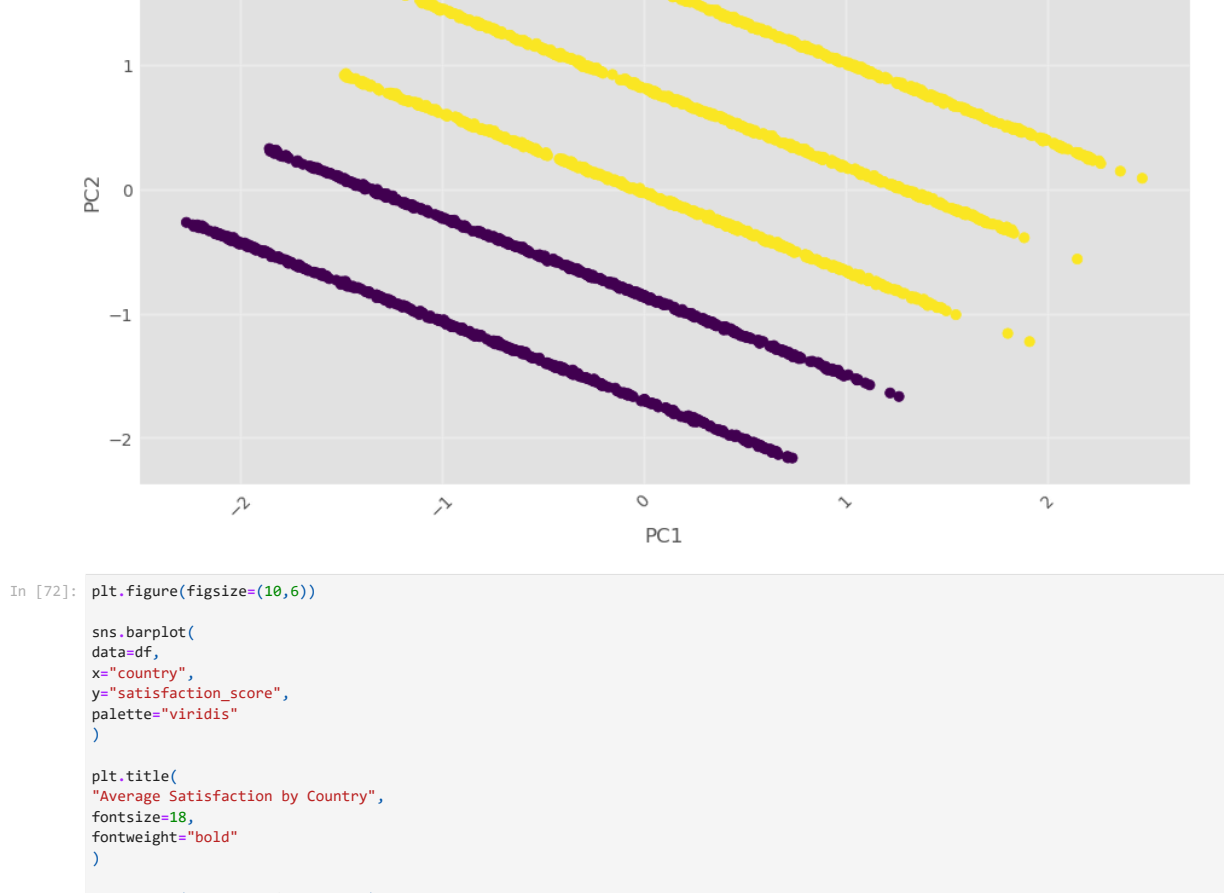
```
[0 1 0 0 1 1 0 0 0 1 1 1 1 1 0 0 0 1 0 0]
```

In [68]:
```python
# add hierarchical labels
df["hierarchical_cluster"] = (hierarchical_labels)
```

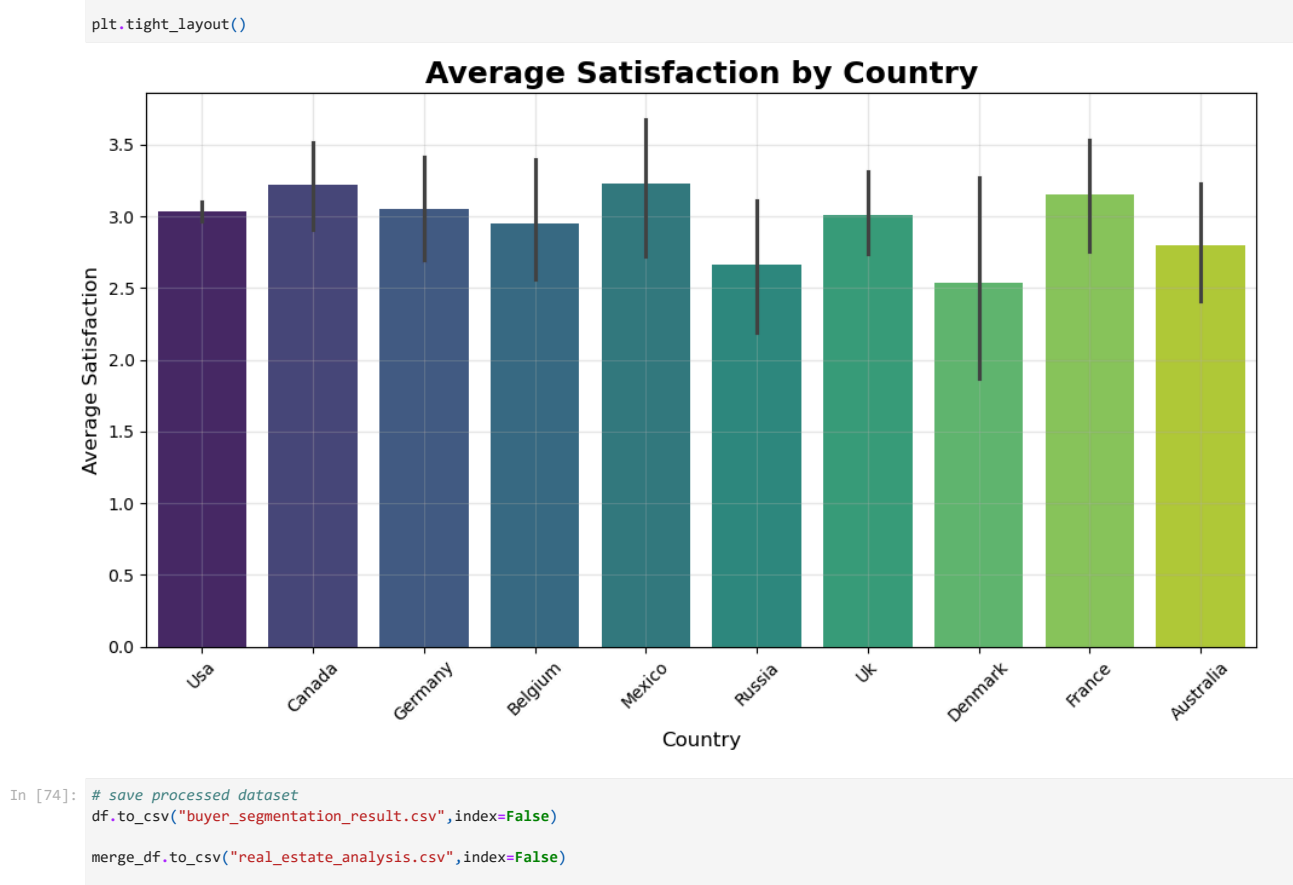
In [69]:
```python
print(df[["client_id","cluster","hierarchical_cluster"]].head(20))
```

```
   client_id  cluster  hierarchical_cluster
0      C0001        1                     0
1      C0002        0                     1
2      C0003        1                     0
3      C0004        1                     0
4      C0005        1                     1
5      C0006        1                     1
6      C0007        1                     0
7      C0008        1                     0
8      C0009        1                     0
9      C0010        1                     1
10     C0011        0                     1
11     C0012        1                     1
12     C0013        0                     1
13     C0014        1                     1
14     C0015        1                     0
15     C0016        1                     0
16     C0017        1                     0
17     C0018        0                     1
18     C0019        1                     0
19     C0020        1                     0
```

In [73]:
```python
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
components =pca.fit_transform(x)
plt.figure(figsize=(10,6))
plt.scatter(components[:,0],components[:,1],c=df["cluster"])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.style.use("fivethirtyeight")
plt.xticks(rotation=45)

plt.style.use("ggplot")
plt.grid(alpha=0.3)
```



In [72]:
```python
plt.figure(figsize=(10,6))

sns.barplot(
data=df,
x="country",
y="satisfaction_score",
palette="viridis"
)

plt.title(
"Average Satisfaction by Country",
fontsize=18,
fontweight="bold"
)

plt.xlabel("Country",fontsize=12)

plt.ylabel("Average Satisfaction",fontsize=12)

plt.style.use("fivethirtyeight")
plt.xticks(rotation=45)

plt.style.use("ggplot")
plt.grid(alpha=0.3)

plt.tight_layout()
```



In [74]:
```python
# save processed dataset
df.to_csv("buyer_segmentation_result.csv",index=False)

merge_df.to_csv("real_estate_analysis.csv",index=False)

print("Files saved successfully!")
```

```
Files saved successfully!
```

In [75]:
```python
joblib.dump(kmeans,"kmeans_model.pkl")

joblib.dump(preprocessor,"preprocessor.pkl")

print("Models saved successfully!")
```

```
Models saved successfully!
```

In [77]:
```python
kmeans.fit(x)
```

Out[77]:

▾ KMeans ⓘ ?

▸ Parameters

▸ Fitted attributes

▸ 2 features

In [78]:
```python
# Craete cluster profile
cluster_profile = (df.groupby("cluster").agg(customers = ("client_id","count"),average_age=("age","mean"),
                                               average_satisfaction = ("satisfaction_score","mean")))
print(cluster_profile)
```

```
         customers  average_age  average_satisfaction
cluster                                              
0              774    55.684755              1.489664
1             1226    56.360522              4.000816
```

In [79]:
```python
# Investment profiling
investment_profile = (merge_df.groupby("cluster").agg(buyers = ("client_id","nunique"),
                                                      total_investment = ("sale_price","sum"),
                                                      average_investment = ("sale_price","mean"),
                                                      average_area = ("floor_area_sqft","mean")))

print(investment_profile)
```

```
         buyers  total_investment  average_investment  average_area
cluster                                                            
0.0         774      9.621043e+08       344716.680677   1139.383963
1.0        1226      1.558647e+09       345291.693635   1142.238440
```

In [80]:
```python
cluster_names = {0:"Global Investors",
                 1:"First-Time buyers",
                 2:"Corporate Buyers",
                 3:"Luxury Investors"}

df["buyer_segment"] = (df["cluster"].map(cluster_names))
```

In [81]:
```python
final_report = (df.groupby(["cluster","buyer_segment"]).agg(customers = ("client_id","count"),
                                                            avg_age = ("age","mean"),
                                                            avg_satisfaction = ("satisfaction_score","mean"),
                                                            investment_buyers = ("acquisition_purpose", lambda x:(x =="yes").sum())).reset_index())

print(final_report)
```

```
   cluster      buyer_segment  customers    avg_age  avg_satisfaction  \
0        0   Global Investors        774  55.684755          1.489664
1        1  First-Time buyers       1226  56.360522          4.000816

   investment_buyers
0                  0
1                  0
```

```
In [82]: pwd
```

```
Out[82]: 'C:\\Users\\mayur'
```